Applied Scientist

Complete Interview Study Guide

Microsoft & Amazon

ML Theory | Deep Learning | Statistics | System Design | Top 100 Q&A

PART 1: Machine Learning Fundamentals

1.1 What is Machine Learning?

Machine Learning (ML) is a field of AI where algorithms learn patterns from data to make predictions or decisions without being explicitly programmed for each scenario.

Three Types of Machine Learning

Supervised Learning	You provide labeled input-output pairs. The model learns to map inputs to outputs. Example: predicting house price (input = sq ft, bedrooms; output = price).
Unsupervised Learning	No labels given. The model finds hidden structure or patterns on its own. Example: grouping customers by purchase behavior (clustering).
Reinforcement Learning	An agent learns by taking actions in an environment, receiving rewards or penalties. Example: training a game-playing AI.

The ML Workflow

1. Collect and clean data
2. Perform Exploratory Data Analysis (EDA)
3. Feature engineering / selection
4. Choose and train a model
5. Evaluate on validation set
6. Tune hyperparameters
7. Test on held-out test set
8. Deploy and monitor

1.2 Bias, Variance & the Tradeoff

These two sources of error determine how well a model generalizes to unseen data.

Bias	Error from wrong assumptions. High bias = model too simple = underfitting. Example: fitting a line to curved data.
Variance	Error from sensitivity to training data noise. High variance = model too complex = overfitting. Example: a deep tree that memorizes training data.

Total Error = Bias^2 + Variance + Irreducible Noise

Tradeoff: As model complexity increases, bias decreases but variance increases. The sweet spot is where total error is minimized.

How to Fix Underfitting (High Bias)

- Add more features or polynomial features
- Increase model complexity (more layers, more trees)
- Reduce regularization

How to Fix Overfitting (High Variance)

- Get more training data
- Add regularization (L1, L2, Dropout)
- Reduce model complexity
- Use cross-validation
- Feature selection / dimensionality reduction

1.3 Regularization

Regularization adds a penalty to the loss function to prevent overfitting by discouraging overly complex models.

L1 (Lasso)	Adds sum of absolute values of weights to loss. Encourages sparsity -- drives many weights to exactly zero. Good for feature selection.

L2 (Ridge)	Adds sum of squared weights to loss. Shrinks weights toward zero but rarely to exactly zero. Better when all features matter somewhat.

Elastic Net	Combination of L1 and L2. Controlled by a mixing ratio. Best of both worlds.

```
# L1 + L2 in Python (sklearn)
from sklearn.linear_model import Ridge, Lasso, ElasticNet

ridge  = Ridge(alpha=1.0)        # L2 regularization
lasso  = Lasso(alpha=0.1)        # L1 regularization
enet   = ElasticNet(alpha=0.1, l1_ratio=0.5)  # Elastic Net

# In PyTorch: add weight_decay to optimizer
optimizer = torch.optim.Adam(model.parameters(), lr=1e-3, weight_decay=1e-4)
```

1.4 Loss Functions

A loss function measures how wrong the model's predictions are. The training process minimizes this loss.

MSE (Regression)	Mean Squared Error. Average of (prediction - actual)^2. Penalizes large errors heavily. Sensitive to outliers.

MAE (Regression)	Mean Absolute Error. Average of |prediction - actual|. More robust to outliers than MSE.

Cross-Entropy (Classification)	Measures difference between predicted probability distribution and actual labels. Used in logistic regression and neural networks.

Huber Loss	Combines MSE for small errors and MAE for large errors. Good when you have outliers but still want smooth gradients.

```
import torch.nn as nn

mse_loss = nn.MSELoss()
mae_loss = nn.L1Loss()
ce_loss  = nn.CrossEntropyLoss()   # for multi-class classification
bce_loss = nn.BCEWithLogitsLoss()  # for binary classification
huber    = nn.HuberLoss(delta=1.0)
```

1.5 Evaluation Metrics

Classification Metrics

Confusion Matrix Terms:

- TP (True Positive): Correctly predicted positive
- FP (False Positive): Predicted positive but actually negative (Type I Error)
- FN (False Negative): Predicted negative but actually positive (Type II Error)
- TN (True Negative): Correctly predicted negative

Accuracy	(TP + TN) / Total. Misleading when classes are imbalanced.

Precision	TP / (TP + FP). Of all predicted positives, how many were actually positive? Optimize when false positives are costly (e.g., spam filter).

Recall (Sensitivity)	TP / (TP + FN). Of all actual positives, how many did we catch? Optimize when false negatives are costly (e.g., cancer detection).

F1 Score	2 * (Precision * Recall) / (Precision + Recall). Harmonic mean. Good for imbalanced datasets.

AUC-ROC	Area Under the ROC Curve. Measures ability to distinguish classes at all thresholds. 1.0 = perfect, 0.5 = random.

Regression Metrics

- MSE: Mean Squared Error -- good default
- RMSE: Root MSE -- same units as target variable

- MAE: Mean Absolute Error -- intuitive, robust to outliers
- R^2 (R-squared): Proportion of variance explained. 1.0 = perfect fit, 0 = baseline mean model

1.6 Cross-Validation

Cross-validation gives a more reliable estimate of model performance by testing on multiple data splits.

K-Fold CV	Split data into K equal folds. Train on K-1 folds, test on 1. Repeat K times. Average the scores. Typical K = 5 or 10.

Stratified K-Fold	Same as K-Fold but preserves class proportions in each fold. Important for imbalanced datasets.

Leave-One-Out (LOO)	K = N (each sample is its own test set). Very accurate but computationally expensive.

```
from sklearn.model_selection import cross_val_score, StratifiedKFold
from sklearn.ensemble import RandomForestClassifier

model = RandomForestClassifier(n_estimators=100)
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
scores = cross_val_score(model, X, y, cv=cv, scoring='f1_macro')
print(f'CV F1: {scores.mean():.3f} +/- {scores.std():.3f}')
```

PART 2: Classical ML Algorithms

2.1 Linear & Logistic Regression

Linear Regression

Predicts a continuous output as a weighted sum of input features. y = w1*x1 + w2*x2 + ... + b

Assumptions: linearity, independence, homoscedasticity (constant variance), normality of residuals.

```
# Linear Regression from scratch
import numpy as np

class LinearRegression:
    def __init__(self, lr=0.01, epochs=1000):
        self.lr = lr; self.epochs = epochs
    def fit(self, X, y):
        n, d = X.shape
        self.w = np.zeros(d); self.b = 0
        for _ in range(self.epochs):
            y_hat = X @ self.w + self.b
            self.w -= self.lr * (2/n) * X.T @ (y_hat - y)
            self.b -= self.lr * (2/n) * np.sum(y_hat - y)
    def predict(self, X):
        return X @ self.w + self.b
```

Logistic Regression

Predicts probability of a class using the sigmoid function: P(y=1) = 1 / (1 + e^(-z)) where z = w*x + b.

Despite the name, it is a CLASSIFICATION algorithm. Uses cross-entropy loss. Decision boundary is linear.

```
import torch, torch.nn as nn

class LogisticRegression(nn.Module):
    def __init__(self, input_dim):
        super().__init__()
        self.linear = nn.Linear(input_dim, 1)
    def forward(self, x):
        return torch.sigmoid(self.linear(x))
```

2.2 Decision Trees

A tree-shaped model that splits data based on feature thresholds. Each internal node is a decision, each leaf is a prediction.

Gini Impurity	Measures how often a randomly chosen element would be incorrectly labeled. Range 0 (pure) to 0.5 (max impurity).

Information Gain	Reduction in entropy after a split. The tree picks the split that maximizes information gain.

Entropy	H = -sum(p * log2(p)). Measures disorder in a node. 0 = perfectly pure, 1 = maximally mixed (50/50 split).

Pros: interpretable, no scaling needed, handles mixed feature types.

Cons: prone to overfitting, unstable (small data changes = big tree changes), biased toward high-cardinality features.

2.3 Ensemble Methods

Combining multiple models almost always outperforms a single model. The key idea: diverse weak learners aggregate into a strong learner.

Random Forest (Bagging)

Builds many decision trees on random subsets of data (bootstrap samples) and random subsets of features. Aggregates by majority vote (classification) or mean (regression).

- Reduces variance without increasing bias
- Each tree sees ~63% of training samples (the rest are out-of-bag)
- Feature importance = average decrease in impurity across trees

Gradient Boosting (Boosting)

Builds trees sequentially. Each tree corrects the residual errors of the previous ensemble. XGBoost, LightGBM, and CatBoost are popular implementations.

- Uses gradient descent in function space
- More prone to overfitting than Random Forest; tune n_estimators and learning_rate carefully
- XGBoost: adds regularization and second-order gradients; faster and more powerful

```
import xgboost as xgb
from sklearn.ensemble import RandomForestClassifier

# Random Forest
rf = RandomForestClassifier(n_estimators=200, max_depth=None,
                            min_samples_leaf=2, random_state=42)

# XGBoost
xgb_model = xgb.XGBClassifier(
    n_estimators=300, learning_rate=0.05,
    max_depth=6, subsample=0.8,
    colsample_bytree=0.8, use_label_encoder=False,
    eval_metric='logloss', random_state=42
)
```

2.4 Support Vector Machines (SVM)

Finds the hyperplane that maximally separates classes. The 'support vectors' are the data points closest to the boundary.

Maximum Margin	SVM maximizes the margin (distance between boundary and nearest points). Larger margin = better generalization.

Kernel Trick	Maps data to a higher-dimensional space where it becomes linearly separable. Common kernels: RBF (Gaussian), Polynomial, Sigmoid. Avoids explicit computation in high dimensions.

C parameter	Controls the trade-off between maximizing margin and minimizing misclassification. High C = smaller margin, less slack (more overfitting). Low C = larger margin, more slack.

Best for: small-to-medium datasets, high-dimensional spaces (text), when a clear margin exists.

2.5 K-Nearest Neighbors (KNN)

A non-parametric algorithm. To predict a new point, find the K nearest training points (by Euclidean or other distance) and take a majority vote (classification) or mean (regression).

- Lazy learner: no training phase, just stores data
- Choosing K: small K = overfitting, large K = underfitting. Use cross-validation.
- Must normalize features -- KNN is distance-based and very sensitive to scale
- O(n) prediction time: slow for large datasets

PART 3: Neural Networks & Deep Learning

3.1 The Perceptron & Neural Network Basics

A neural network is a composition of layers of artificial neurons. Each neuron computes a weighted sum of inputs and applies an activation function.

Neuron	output = activation(w1*x1 + w2*x2 + ... + wn*xn + b). The weights w and bias b are learned parameters.

Layer	A group of neurons. Input layer receives raw data. Hidden layers extract features. Output layer produces predictions.

Universal Approximation	A neural network with at least one hidden layer and enough neurons can approximate any continuous function. This is the theoretical basis for why deep learning works.

Activation Functions

ReLU	f(x) = max(0, x). Most popular. Avoids vanishing gradient. Problem: 'dying ReLU' if neuron always outputs 0. Use for hidden layers.

Sigmoid	f(x) = 1/(1+e^-x). Outputs 0 to 1. Use for binary output layer. Causes vanishing gradients in deep networks.

Tanh	f(x) = (e^x - e^-x)/(e^x + e^-x). Outputs -1 to 1. Zero-centered; better than sigmoid in hidden layers. Still has vanishing gradient issue.

Softmax	Converts logits to probabilities summing to 1. Use for multi-class output layer.

LeakyReLU / ELU	Fixes dying ReLU by allowing small negative outputs for x < 0.

```
import torch.nn as nn
import torch.nn.functional as F

# Simple 3-layer neural network
class MLP(nn.Module):
    def __init__(self, input_dim, hidden_dim, output_dim):
        super().__init__()
        self.fc1 = nn.Linear(input_dim, hidden_dim)
        self.fc2 = nn.Linear(hidden_dim, hidden_dim)
        self.fc3 = nn.Linear(hidden_dim, output_dim)
        self.dropout = nn.Dropout(0.3)
    def forward(self, x):
        x = F.relu(self.fc1(x))
        x = self.dropout(x)
        x = F.relu(self.fc2(x))
```

```
        x = self.fc3(x)    # no activation here; use with CrossEntropyLoss
        return x
```

3.2 Backpropagation & Gradient Descent

Backpropagation computes gradients of the loss with respect to each weight using the chain rule of calculus. The gradients tell us how to adjust weights to reduce loss.

Gradient Descent Variants

Batch GD	Uses ALL training data to compute one gradient update. Stable but slow and memory-intensive.

Stochastic GD (SGD)	Uses ONE random sample per update. Fast but noisy. The noise can help escape local minima.

Mini-batch GD	Uses a small batch (e.g., 32 or 128 samples). Best of both worlds -- the standard in deep learning.

Optimizers

SGD + Momentum	Accumulates velocity in gradient direction. Helps cross saddle points and reduces oscillations.

Adam	Adaptive Moment Estimation. Maintains per-parameter learning rates. Combines Momentum + RMSProp. Default choice for most problems.

AdamW	Adam with decoupled weight decay. Often outperforms Adam for transformers. Default for LLM training.

```
import torch.optim as optim

optimizer = optim.Adam(model.parameters(), lr=1e-3, betas=(0.9, 0.999), eps=1e-8)

# Training loop
for epoch in range(100):
    for X_batch, y_batch in dataloader:
        optimizer.zero_grad()          # clear gradients
        y_pred = model(X_batch)        # forward pass
        loss = criterion(y_pred, y_batch)  # compute loss
        loss.backward()                # backprop
        optimizer.step()               # update weights
```

3.3 Convolutional Neural Networks (CNNs)

CNNs are specialized for grid-like data such as images. They use convolution operations to extract local patterns (edges, textures, objects).

Convolution	A filter/kernel slides over the input, computing dot products at each position. Detects local features regardless of position (translation equivariance).

Pooling	Downsamples feature maps. MaxPooling takes the maximum value in a window. Reduces spatial dimensions and adds translation invariance.

Feature Map	The output of applying a filter to the input. Each filter detects a different pattern.

Receptive Field	The region of input that affects a particular neuron. Gets larger with depth.

Key CNN Architectures

- LeNet-5 (1998): First successful CNN. Digits recognition.
- AlexNet (2012): Won ImageNet. Introduced ReLU and Dropout at scale.
- VGG-16/19: Stacked 3x3 convolutions. Simple and effective.
- ResNet (2015): Introduced skip/residual connections. Solved vanishing gradient in very deep nets. Still widely used.
- EfficientNet: Scales width, depth, and resolution together. State-of-the-art efficiency.

```
class SimpleCNN(nn.Module):
    def __init__(self, num_classes=10):
        super().__init__()
        self.conv1 = nn.Conv2d(3, 32, kernel_size=3, padding=1)
        self.conv2 = nn.Conv2d(32, 64, kernel_size=3, padding=1)
        self.pool  = nn.MaxPool2d(2, 2)
        self.fc1   = nn.Linear(64 * 8 * 8, 256)
        self.fc2   = nn.Linear(256, num_classes)
    def forward(self, x):         # x: (B, 3, 32, 32)
        x = self.pool(F.relu(self.conv1(x)))  # -> (B, 32, 16, 16)
        x = self.pool(F.relu(self.conv2(x)))  # -> (B, 64, 8, 8)
        x = x.flatten(1)                       # -> (B, 4096)
        x = F.relu(self.fc1(x))
        return self.fc2(x)
```

3.4 Recurrent Neural Networks (RNNs) & LSTMs

RNNs process sequential data by maintaining a hidden state that captures context from previous steps. Used for text, time series, speech.

Vanishing Gradient	In deep/long sequences, gradients shrink exponentially during backprop through time. Early steps get no useful gradient signal.

LSTM	Long Short-Term Memory. Solves vanishing gradient with gating mechanisms: Forget Gate (what to erase), Input Gate (what to write), Output Gate (what to read). The cell state acts as long-term memory.

GRU	Gated Recurrent Unit. Simpler than LSTM (only 2 gates). Often comparable performance. Faster to train.

```
class LSTMModel(nn.Module):
    def __init__(self, input_size, hidden_size, num_layers, output_size):
        super().__init__()
        self.lstm = nn.LSTM(input_size, hidden_size, num_layers, batch_first=True)
        self.fc = nn.Linear(hidden_size, output_size)
    def forward(self, x):              # x: (batch, seq_len, input_size)
        out, (h_n, c_n) = self.lstm(x)
        return self.fc(out[:, -1, :]) # use last time step
```

3.5 Transformers & Attention Mechanism

Transformers are the architecture behind BERT, GPT, and modern LLMs. They replaced RNNs for most NLP tasks due to parallelism and the ability to capture long-range dependencies.

Self-Attention

For each token, compute attention scores against all other tokens. Allows the model to focus on relevant parts of the input regardless of distance.

```
Attention(Q, K, V) = softmax(QK^T / sqrt(d_k)) * V
```

Q (Query)	What the current token is looking for.

K (Key)	What each token is advertising about itself.

V (Value)	What information each token will contribute if selected.

Multi-Head Attention	Run attention multiple times in parallel with different learned projections. Each head can attend to different aspects. Outputs are concatenated and projected.

Transformer Architecture

- Encoder: Stack of self-attention + FFN layers. Builds rich contextual representations. (BERT)
- Decoder: Like encoder but also has cross-attention over encoder output. Generates output tokens. (GPT)
- Positional Encoding: Since attention has no notion of order, we add positional embeddings to input tokens.
- Layer Norm: Applied before (Pre-LN) or after (Post-LN) each sub-layer. Stabilizes training.

3.6 Normalization Techniques

Batch Normalization	Normalizes activations across the batch dimension. Stabilizes training, allows higher learning rates. Works well for CNNs. Issue: depends on batch size; behaves differently at train/inference.

Layer Normalization	Normalizes across the feature dimension for each sample independently. Works well for RNNs and Transformers. Not dependent on batch size.

Dropout	Randomly sets a fraction of neurons to zero during training. Forces network to learn redundant representations. Acts as regularization. Rate typically 0.1-0.5.

PART 4: Statistics & Probability

4.1 Probability Fundamentals

Conditional Probability	P(A|B) = P(A and B) / P(B). Probability of A given B has occurred.

Bayes' Theorem	P(A|B) = P(B|A) * P(A) / P(B). Posterior = Likelihood * Prior / Evidence. Foundation of Bayesian ML.

Independence	Events A and B are independent if P(A and B) = P(A)*P(B). Knowing B gives no information about A.

Key Distributions

- Normal (Gaussian): Bell curve. Defined by mean and variance. Central Limit Theorem: mean of many samples tends toward Normal.
- Bernoulli: Single coin flip. P(X=1) = p, P(X=0) = 1-p.
- Binomial: Number of successes in N Bernoulli trials.
- Poisson: Count of events in a fixed interval. Used for rare events (clicks, arrivals).
- Exponential: Time between Poisson events.

4.2 Hypothesis Testing

Used to determine if an observed effect is statistically significant or due to chance.

Null Hypothesis (H0)	The default assumption: no effect or no difference.

p-value	Probability of observing data as extreme as yours if H0 is true. Low p-value (< 0.05) = reject H0.

Type I Error	Rejecting H0 when it is actually true. False positive. Controlled by significance level alpha (typically 0.05).

Type II Error	Failing to reject H0 when it is actually false. False negative. Related to statistical power.

A/B Testing	Controlled experiment comparing two versions (A = control, B = treatment). Essential for product decisions at Amazon/Microsoft. Use t-test or chi-squared test for significance.

4.3 Descriptive Statistics

- Mean: Sum / N. Sensitive to outliers.
- Median: Middle value when sorted. Robust to outliers. Better for skewed data.
- Mode: Most frequent value.
- Variance: Average squared deviation from the mean. sigma^2 = E[(X - mu)^2]
- Standard Deviation: sqrt(Variance). Same units as data.
- Covariance: Measures how two variables change together. Positive = same direction.
- Correlation: Normalized covariance. Range [-1, 1]. Pearson for linear; Spearman for rank-based.

4.4 Maximum Likelihood Estimation (MLE)

MLE finds the parameter values that maximize the probability (likelihood) of observing the training data. Training a logistic regression with cross-entropy = MLE under a Bernoulli distribution.

```
log L(theta) = sum_i log P(y_i | x_i; theta)  -- maximize this
```

PART 5: Feature Engineering & Data Preprocessing

5.1 Handling Missing Data

Drop rows/cols	Remove if missing rate is high (>50%) or if data is missing at random. Risk: information loss.
Mean/Median Imputation	Replace with column average or median. Simple but may distort distributions.
Model-based Imputation	Use KNN or regression to predict missing values based on other features. More accurate.
Flag + Impute	Create a binary indicator column for 'was this value missing?' then impute. The missingness itself can be informative.

5.2 Feature Scaling

Many ML algorithms (KNN, SVM, neural networks, PCA) are sensitive to feature scales. Tree-based models do NOT need scaling.

Min-Max Scaling	x_scaled = (x - min) / (max - min). Output range [0, 1]. Sensitive to outliers.
Standardization (Z-score)	x_scaled = (x - mean) / std. Output has mean=0, std=1. More robust to outliers. Use for most ML.
Robust Scaler	Uses median and IQR instead of mean and std. Best when many outliers present.

5.3 Encoding Categorical Variables

One-Hot Encoding	Creates binary column for each category. Use for nominal (unordered) categories. Risk: high dimensionality for many categories.
Label Encoding	Assigns integer to each category (0, 1, 2...). Only use when order is meaningful (ordinal data). Otherwise implies false ordering.
Target Encoding	Replace category with mean of target variable for that category. Powerful but prone to overfitting -- use cross-validation.
Embeddings	For NLP/high-cardinality categoricals: learn a dense vector representation. Used in deep learning.

5.4 Dimensionality Reduction

PCA (Principal Component Analysis)

Finds orthogonal directions (principal components) of maximum variance. Projects data to lower dimension while preserving maximum variance. Great for visualization, noise reduction, removing correlated features.

```
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler

# Always scale before PCA
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

pca = PCA(n_components=0.95)  # keep 95% of variance
X_pca = pca.fit_transform(X_scaled)
print(f'Components: {pca.n_components_}')  # number of components used
```

t-SNE & UMAP

Non-linear dimensionality reduction, primarily for visualization. t-SNE: preserves local structure; good for seeing clusters. UMAP: faster than t-SNE; better preserves global structure.

PART 6: Natural Language Processing (NLP)

6.1 Text Preprocessing

- Tokenization: Split text into words or subwords
- Lowercasing, removing punctuation/stop words
- Stemming: Reduce to word root (running -> run)
- Lemmatization: Reduce to dictionary form (better -> good)
- TF-IDF: Term Frequency * Inverse Document Frequency. Weights words by importance across documents

6.2 Word Embeddings

Word2Vec	Neural model that learns dense word vectors. Similar words have similar vectors. Skip-gram: predict context from word. CBOW: predict word from context.
GloVe	Global Vectors for Word Representation. Factorizes word co-occurrence matrix. Captures global statistics.
Contextual Embeddings	BERT, GPT: same word gets different vectors depending on context. 'Bank' in 'river bank' vs 'savings bank' gets different representations.

6.3 BERT & GPT

BERT	Bidirectional Encoder Representations from Transformers. Encoder-only. Pre-trained with Masked Language Model (MLM) and Next Sentence Prediction. Best for understanding tasks: classification, NER, QA.
GPT	Generative Pre-trained Transformer. Decoder-only. Trained with causal (left-to-right) language modeling. Best for generation tasks: text completion, summarization, dialogue.
Fine-tuning	Take a pre-trained model (BERT/GPT) and continue training on task-specific labeled data. Much faster than training from scratch. Foundation of modern NLP.

6.4 Named Entity Recognition (NER) & Text Classification

NER: Identify and classify named entities (persons, organizations, locations) in text. Text Classification: Assign a category to a document (sentiment, topic, intent).

```
from transformers import pipeline


# Sentiment analysis with BERT
```

```
classifier = pipeline('sentiment-analysis')
result = classifier('I love this product!')
# [{'label': 'POSITIVE', 'score': 0.9998}]

# NER
ner = pipeline('ner', grouped_entities=True)
result = ner('Amazon was founded by Jeff Bezos in Seattle.')
```

PART 7: ML System Design

7.1 The ML System Design Framework

In interviews, use this structured approach:

9. Clarify requirements (scope, scale, constraints, latency SLA)
10. Define the ML problem (what to predict? what label?)
11. Data collection and annotation strategy
12. Feature engineering
13. Model selection and training
14. Evaluation metrics (offline + online)
15. Serving and deployment (batch vs. real-time)
16. Monitoring and feedback loops

7.2 Example: Design a Recommendation System

Collaborative Filtering

Recommend items that similar users liked. No item content needed.

- User-based: find similar users, recommend what they liked
- Item-based: find similar items to what user liked (more scalable)
- Matrix Factorization (SVD, ALS): decompose user-item interaction matrix into latent factors

Content-Based Filtering

Recommend items similar in features to what user liked. Requires item metadata.

Two-Tower Model (Industry Standard)

Train a user encoder and an item encoder separately. At inference, find items whose embeddings are closest to user embedding using Approximate Nearest Neighbor (FAISS, ScaNN).

7.3 Training vs. Serving Skew

One of the most common production ML problems: the model sees different data distribution during serving than during training.

- Cause: using training data statistics (mean, std) computed on full dataset for serving
- Cause: feature engineering logic differs between training pipeline and serving pipeline
- Fix: use the same feature pipeline (e.g., TF Transform) for both training and serving

7.4 Data Imbalance

- Oversampling minority class (SMOTE: generates synthetic samples)
- Undersampling majority class

- Class weights in loss function: loss *= class_weight[y]
- Use PR-AUC or F1 instead of accuracy for evaluation

7.5 Model Monitoring

Data Drift	Input feature distribution changes over time. Detect with statistical tests (KS test, PSI).
Concept Drift	The relationship between features and target changes over time. Requires retraining.
Model Degradation	Performance metrics degrade in production. Monitor AUC, precision, recall on live traffic.

PART 8: Advanced Topics

8.1 Transfer Learning

Use a model pre-trained on a large dataset (ImageNet, Common Crawl) as a starting point. Fine-tune on your smaller task-specific dataset.

- Feature extraction: freeze pre-trained layers, only train a new head
- Fine-tuning: unfreeze some/all layers and train with a small learning rate
- Benefit: requires much less labeled data and compute

8.2 Hyperparameter Tuning

Grid Search	Try all combinations of a predefined grid. Exhaustive but exponential in number of hyperparameters.

Random Search	Sample random combinations. Surprisingly effective; often better than Grid Search for same compute budget.

Bayesian Optimization	Build a probabilistic model of hyperparameter -> performance. Choose next parameters based on expected improvement. Optuna, HyperOpt.

8.3 Generative Models

VAE (Variational Autoencoder)	Encoder maps input to a latent distribution (mean + std). Decoder reconstructs from sampled latent vector. Smooth latent space enables interpolation.

GAN (Generative Adversarial Network)	Generator creates fake samples; Discriminator tries to distinguish real from fake. They are trained adversarially. Generator improves until discriminator can't tell the difference.

Diffusion Models	Add Gaussian noise to data iteratively, then train a model to reverse the process. Foundation of DALL-E, Stable Diffusion. State-of-the-art for image generation.

8.4 Reinforcement Learning Key Concepts

MDP	Markov Decision Process: (State, Action, Reward, Next State, Discount factor gamma). The formal framework for RL.

Q-Learning	Learn Q(s, a) = expected cumulative reward from state s, taking action a. Off-policy. Bellman equation: Q(s,a) = R + gamma * max_a' Q(s', a').

Policy Gradient	Directly optimize the policy (action distribution). REINFORCE, PPO, A3C. Used in ChatGPT's RLHF training.

PART 9: Top 100 Interview Questions & Answers

These questions cover what Microsoft and Amazon Applied Scientist interviews focus on: ML theory, coding, system design, and behavioral.

Q1. What is the difference between classification and regression?

Classification predicts a discrete category (e.g., spam/not spam). Regression predicts a continuous value (e.g., house price). Different loss functions: cross-entropy for classification, MSE/MAE for regression.

Q2. What is overfitting and how do you prevent it?

Overfitting is when a model learns training data noise and performs poorly on new data (high variance). Prevent with: more data, regularization (L1/L2/Dropout), cross-validation, reducing model complexity, early stopping.

Q3. Explain bias-variance tradeoff.

Bias = error from wrong assumptions (underfitting). Variance = error from sensitivity to training noise (overfitting). Total error = Bias^2 + Variance + irreducible noise. Increasing complexity decreases bias but increases variance.

Q4. What is the difference between L1 and L2 regularization?

L1 (Lasso) adds sum of |weights| to loss. Produces sparse models (some weights = 0), useful for feature selection. L2 (Ridge) adds sum of weights^2. Shrinks all weights but rarely to zero. L1 is better when you suspect many irrelevant features.

Q5. Why is accuracy a bad metric for imbalanced datasets?

If 99% of data is class 0, a model that always predicts class 0 gets 99% accuracy but is useless. Better metrics: Precision, Recall, F1-score, AUC-ROC, or PR-AUC, which properly measure performance on the minority class.

Q6. Explain gradient descent and its variants.

Gradient descent minimizes loss by iteratively moving parameters in the direction of the negative gradient. Batch GD: uses all data per update (slow). SGD: uses 1 sample (noisy but fast). Mini-batch GD: uses a batch of 32-512 samples (standard in deep learning).

Q7. What is the vanishing gradient problem?

In deep networks with sigmoid/tanh activations, gradients become extremely small during backpropagation, making early layers learn very slowly or not at all. Solutions: ReLU activations, residual connections (ResNet), batch normalization, gradient clipping.

Q8. Explain the attention mechanism in transformers.

Attention computes a weighted sum of values, where weights are derived from the similarity between a query and set of keys. Self-attention: each token attends to all other tokens. Formula: softmax(QK^T/sqrt(d_k))V. Enables modeling long-range dependencies.

Q9. What is cross-validation and why do we use it?

K-Fold CV splits data into K folds, trains on K-1, tests on 1, and repeats K times. We use it to get a reliable estimate of model performance on unseen data, especially when data is limited. Helps with hyperparameter tuning without contaminating the test set.

Q10. What is the difference between bagging and boosting?

Bagging trains multiple independent models on bootstrap samples and averages their predictions (e.g., Random Forest). Reduces variance. Boosting trains models sequentially, each correcting previous errors (e.g., XGBoost). Reduces both bias and variance. Boosting typically achieves higher accuracy but is more prone to overfitting.

Q11. What is backpropagation?

An algorithm that computes the gradient of the loss with respect to each weight using the chain rule of calculus. Starting from the output layer, gradients are propagated backward through the network. These gradients are then used by an optimizer (e.g., Adam) to update weights.

Q12. Why use ReLU instead of sigmoid in hidden layers?

ReLU (max(0,x)) does not saturate for positive values, so gradients flow freely. Sigmoid saturates at both ends, causing vanishing gradients in deep networks. ReLU is also computationally cheaper (simple threshold). Drawback: dying ReLU (neuron stuck at 0 if always negative).

Q13. Explain batch normalization.

Normalizes activations within a mini-batch to have mean=0, std=1, then applies learnable scale and shift. Benefits: enables higher learning rates, reduces dependence on initialization, acts as regularization. Applied before or after activation. Issue: behavior differs at train vs. inference time.

Q14. What are residual connections (skip connections)?

Introduced in ResNet. Instead of learning H(x), the layer learns the residual F(x) = H(x) - x, and output = F(x) + x. This creates shortcut paths that allow gradients to flow directly, enabling training of very deep networks (100+ layers) without vanishing gradients.

Q15. What is dropout and how does it work?

During training, each neuron is randomly set to 0 with probability p (typically 0.2-0.5). This prevents co-adaptation of neurons and forces the network to learn redundant, robust representations. At inference, all neurons are active but weights are scaled by (1-p) to maintain expected activation values.

Q16. What is the difference between BERT and GPT?

BERT is encoder-only, bidirectional, pre-trained with masked language modeling. Best for understanding tasks (classification, NER, QA). GPT is decoder-only, unidirectional (left-to-right), pre-trained with causal language modeling. Best for generation tasks. Both are transformer-based.

Q17. Explain word embeddings.

Dense vector representations of words where semantically similar words have similar vectors. Word2Vec (skip-gram/CBOW) and GloVe learn static embeddings. BERT/GPT generate contextual embeddings where the same word gets different vectors depending on context.

Q18. What is multi-head attention?

Instead of one attention operation, we run attention h times in parallel with different learned projections (heads). Each head can capture different types of relationships (syntax, semantics, coreference). Outputs are concatenated and projected. Allows the model to jointly attend to information from different subspaces.

Q19. How do you choose the number of hidden layers and neurons?

Start with a proven architecture for the task. Wider layers = more capacity per layer (can capture complex functions). Deeper networks = more compositional abstraction. Use cross-validation to tune. Rules of thumb: 1-3 layers for tabular data; deeper for images/NLP. Monitor training vs validation loss.

Q20. What is an autoencoder?

Neural network with an encoder (compresses input to latent code) and decoder (reconstructs input from code). Trained to minimize reconstruction error. Uses: dimensionality reduction, anomaly detection (high reconstruction error = anomaly), denoising, pre-training. Variational Autoencoders add probabilistic latent space.

Q21. Explain the Central Limit Theorem (CLT).

The CLT states that the distribution of the sample mean approaches a Normal distribution as sample size increases, regardless of the original data distribution. Critical for hypothesis testing and confidence intervals. Typically n > 30 is sufficient.

Q22. What is a p-value?

The probability of observing data as extreme as the collected sample, assuming the null hypothesis is true. A low p-value (typically < 0.05) suggests the null hypothesis is unlikely and we reject it. Common misconception: p-value is NOT the probability that H0 is true.

Q23. What is the difference between Type I and Type II error?

Type I error (false positive): reject H0 when it is true. Controlled by significance level alpha. Type II error (false negative): fail to reject H0 when it is false. Controlled by increasing sample size or effect size. In medicine: Type I = false alarm. Type II = missed diagnosis.

Q24. Explain A/B testing.

A controlled experiment comparing control (A) and treatment (B) versions. Randomly assign users, measure metrics, use statistical tests (t-test, chi-squared) to determine if differences are significant. Key concerns: sample size (power analysis), novelty effects, multiple comparisons, network effects.

Q25. What is Bayes' Theorem?

P(A|B) = P(B|A)*P(A)/P(B). Posterior = Likelihood * Prior / Evidence. Allows updating beliefs with new data. Example: P(spam|word) = P(word|spam)*P(spam)/P(word). Foundational to Bayesian inference, Naive Bayes classifier, and probabilistic programming.

Q26. What is covariance vs. correlation?

Covariance measures the direction of linear relationship between two variables (can be any magnitude). Correlation normalizes covariance to [-1, 1]: Pearson correlation = Cov(X,Y) / (std_X * std_Y). Correlation is unitless and easier to interpret. 0 = no linear relationship; +/-1 = perfect linear relationship.

Q27. What is MLE (Maximum Likelihood Estimation)?

Finding parameter values theta that maximize P(data | theta) -- the probability of observing the training data. For Gaussian data, MLE of mean = sample mean. For logistic regression, MLE = minimizing cross-entropy loss. MLE can overfit with small data; use MAP (Maximum a Posteriori) to add a prior.

Q28. Explain the curse of dimensionality.

As dimensions increase, data becomes sparse: the volume of the space grows exponentially. Most ML algorithms rely on similarity/distance, which breaks down in high dimensions. Distances become nearly equal for all points. Solutions: PCA, feature selection, embeddings, regularization.

Q29. What is bootstrapping?

Resampling technique: create many datasets by sampling with replacement from original data, compute statistic on each. Used to estimate confidence intervals and standard errors when analytical solutions are unavailable. Foundation of bagging/Random Forest.

Q30. What is the difference between parametric and non-parametric models?

Parametric: assumes a specific functional form (e.g., linear regression assumes linearity). Fixed number of parameters regardless of data size. Non-parametric: makes fewer assumptions; model complexity grows with data (e.g., KNN, decision trees, Gaussian processes). Non-parametric models are more flexible but need more data.

Q31. How does random forest handle missing values?

Scikit-learn's RF does not handle NaN natively -- you must impute first. More advanced implementations (XGBoost, LightGBM) can handle missing values by learning the optimal direction for missing data at each split.

Q32. What is the kernel trick in SVM?

Implicitly maps data to a high-dimensional space via a kernel function K(x, z) = phi(x).phi(z), without explicitly computing the transformation. The RBF kernel effectively maps to infinite-dimensional space. Allows linear classifiers to learn non-linear boundaries efficiently.

Q33. Explain the difference between generative and discriminative models.

Discriminative (Logistic Regression, SVM, NNs): learns P(y|x) directly. Better classification accuracy. Generative (Naive Bayes, GMM, VAE): models P(x, y) = P(x|y)*P(y). Can generate new samples. Handles missing data naturally. Generally needs more data to match discriminative performance.

Q34. What is the EM algorithm?

Expectation-Maximization: iterative algorithm for MLE with latent variables. E-step: compute expected log-likelihood using current parameters (soft assignments). M-step: maximize to find new parameters. Converges to local optimum. Used in Gaussian Mixture Models, Hidden Markov Models.

Q35. How does PCA work?

Finds orthogonal directions (principal components) of maximum variance via eigendecomposition of the covariance matrix (or SVD). Projects data onto the top-k components. Always standardize data before PCA. Useful for dimensionality reduction, visualization, noise removal, removing multicollinearity.

Q36. What is DBSCAN?

Density-Based Spatial Clustering of Applications with Noise. Groups together points with many nearby neighbors (core points), reachable points, and labels isolated points as noise (-1). Advantages: finds arbitrary-shaped clusters, robust to outliers, does not need to specify K. Sensitive to epsilon and min_samples parameters.

Q37. Explain gradient boosting step by step.

1) Initialize with constant prediction (mean of y). 2) Compute residuals (errors). 3) Fit a shallow decision tree to the residuals. 4) Add scaled tree predictions to current model. 5) Recompute residuals. 6) Repeat steps 3-5. Result: ensemble of trees where each new tree corrects previous errors.

Q38. What is Naive Bayes and why is it 'naive'?

Naive Bayes is a probabilistic classifier using Bayes' theorem. It is 'naive' because it assumes all features are conditionally independent given the class -- which is usually false but works surprisingly well for text classification. Fast, requires little data, handles high dimensions well.

Q39. How do you detect and handle outliers?

Detection: Z-score (|z| > 3), IQR method (< Q1 - 1.5*IQR or > Q3 + 1.5*IQR), Isolation Forest, DBSCAN noise points. Handling: remove if data entry error; cap (winsorization) if real but extreme; use robust models (tree-based, MAE loss); create 'is_outlier' feature.

Q40. What is collaborative filtering?

Recommendation technique that predicts a user's preferences based on the behavior of similar users (user-based) or similar items (item-based), without using item content. Matrix factorization (SVD, ALS) learns latent user and item factors. Cold-start problem: new users/items with no history.

Q41. How would you implement K-Means clustering?

1) Randomly initialize K cluster centroids. 2) Assign each point to the nearest centroid (E-step). 3) Update centroids as mean of assigned points (M-step). 4) Repeat steps 2-3 until convergence. Choose K with elbow method (inertia vs K) or silhouette score. K-Means++ for better initialization.

Q42. Explain how you would build a text classifier.

1) Preprocess: tokenize, lowercase, remove stop words. 2) Represent text: TF-IDF for simple models; BERT embeddings for deep learning. 3) Model: start with Logistic Regression or SVM baseline; upgrade to fine-tuned BERT for better accuracy. 4) Evaluate: accuracy, F1. 5) Handle class imbalance if needed.

Q43. How do you handle class imbalance?

Options: 1) Oversampling minority class (SMOTE creates synthetic samples). 2) Undersampling majority class. 3) Class weights in loss function (sklearn class_weight='balanced'). 4) Ensemble methods (BalancedRandomForest). 5) Use appropriate metric (F1, AUC, PR-AUC instead of accuracy).

Q44. What are the steps for feature selection?

1) Filter methods: statistical tests (correlation, chi-squared, mutual information) -- fast. 2) Wrapper methods: forward/backward selection using model performance -- slow but effective. 3) Embedded methods: L1 regularization, Random Forest feature importances -- best of both. Always check for data leakage.

Q45. How would you detect data leakage?

Data leakage occurs when training data contains information about the target that would not be available at prediction time. Signs: suspiciously high CV scores. Prevention: split data before preprocessing, never compute statistics on full dataset then train, use pipelines in sklearn to ensure proper sequencing.

Q46. Explain how you would perform hyperparameter tuning.

1) Start with a reasonable baseline. 2) Use RandomizedSearchCV for a broad search (faster than grid search). 3) Narrow down with GridSearchCV around best area. 4) For deep learning: use Optuna (Bayesian optimization). Always tune on validation set; evaluate final model once on test set.

Q47. How do you deploy a machine learning model?

1) Serialize model (pickle, joblib, ONNX, TorchScript). 2) Build an API (Flask/FastAPI). 3) Containerize (Docker). 4) Deploy to cloud (AWS SageMaker, Azure ML, GCP Vertex AI). 5) Set up monitoring (data drift, model performance). 6) Implement A/B testing for gradual rollout. 7) Set up retraining pipeline.

Q48. What is feature importance in Random Forest?

Measures how much each feature decreases impurity (Gini/entropy) across all trees. Higher = more important. Limitations: biased toward high-cardinality features; correlated features split importance.

Alternative: permutation importance (how much performance drops when feature is shuffled) -- more reliable.

Q49. How would you design an experiment to evaluate a new model?

1) Define clear success metrics (business + ML). 2) Establish baseline. 3) Use offline evaluation: holdout test set or time-based split. 4) Launch shadow mode (new model runs but doesn't serve). 5) A/B test on small % of traffic. 6) Monitor for latency, errors, business metrics. 7) Gradual rollout.

Q50. What is SMOTE?

Synthetic Minority Over-sampling Technique. Creates synthetic minority class examples by interpolating between existing minority samples and their K nearest neighbors. Better than random oversampling as it does not just duplicate existing samples. Apply only to training data (never test).

Q51. Design a fraud detection system.

Problem: binary classification (fraud/not). Features: transaction amount, location, time, device, historical behavior. Model: gradient boosting (XGBoost) for tabular; real-time features from feature store. Threshold tuning: high recall (miss fewer frauds). Deployment: sub-100ms latency API. Monitor: precision/recall drift, new fraud patterns.

Q52. Design a search ranking system.

Two-stage: 1) Retrieval: BM25 or dense retrieval (bi-encoder) to get top-1000 candidates efficiently. 2) Reranking: cross-encoder or LambdaRank/XGBoost trained on click data (learning-to-rank). Features: query-document similarity, CTR, freshness, authority. Metric: NDCG, MRR.

Q53. Design a spam email classifier.

Features: sender reputation, email metadata, content TF-IDF, URL analysis. Model: Naive Bayes baseline, then gradient boosting or BERT. Key metric: high precision (minimize false positives -- losing good emails is bad). Adversarial: spammers adapt, so continuous monitoring and retraining needed.

Q54. Explain the two-tower model for recommendations.

User tower: encodes user history, demographics into user embedding. Item tower: encodes item features into item embedding. Similarity score = dot product of embeddings. Trained on positive interactions (clicks, purchases) with negative sampling. ANN search (FAISS) retrieves top-K items at inference. Scalable and industry-standard.

Q55. How would you handle cold start in recommendations?

New user: use demographic info, onboarding survey, popular items, or explore-exploit bandit. New item: use content-based features (title, description, category) to find similar items. Hybrid approach: content-based for cold start, collaborative filtering as data accumulates.

Q56. What is the difference between online and batch serving?

Batch: precompute predictions for all inputs periodically (e.g., daily recommendations). Low latency, no real-time data. Online/real-time: compute prediction on-demand per request. Uses latest data, handles cold start. Higher infrastructure cost. Shadow mode: run new model online but use old model's predictions.

Q57. How do you ensure fairness in ML models?

1) Audit data for representation bias. 2) Measure fairness metrics per demographic group (equal opportunity, demographic parity). 3) Adversarial debiasing. 4) Reweighting / resampling underrepresented groups. 5) Post-processing: adjust thresholds per group. 6) Regular bias audits in production.

Q58. What is a feature store?

A centralized repository for storing, computing, and serving features consistently across training and serving. Solves train-serve skew. Key components: offline store (for training), online store (for low-latency serving), transformation logic. Examples: Feast, Tecton, Amazon SageMaker Feature Store.

Q59. What is a data pipeline for ML?

ETL (Extract, Transform, Load) workflow that: 1) collects raw data from sources (databases, logs, APIs), 2) cleans and preprocesses (handle missing values, encode categoricals), 3) computes features, 4) stores in a feature store or data warehouse, 5) feeds into training and serving. Tools: Spark, Airflow, Beam, dbt.

Q60. Explain the trade-off between model complexity and interpretability.

Simple models (linear regression, decision trees): interpretable but limited capacity. Complex models (deep networks, ensembles): high accuracy but black-box. Use SHAP or LIME to explain complex model predictions locally. At Microsoft/Amazon: sometimes interpretability is a requirement (regulatory, legal). Start simple and justify complexity.

Q61. What is transfer learning and when should you use it?

Use a model pre-trained on large data as a starting point. Benefits: less labeled data needed, faster training, better generalization. When to use: when you have limited data, similar domain to pre-trained model. Fine-tune: freeze early layers (general features), train later layers (task-specific).

Q62. What is RLHF (Reinforcement Learning from Human Feedback)?

Technique used to align LLMs with human preferences (used in ChatGPT). 1) Fine-tune LLM on demonstrations. 2) Train a reward model from human preference comparisons. 3) Use PPO to optimize the LLM policy to maximize the reward model's score while staying close to the original model. Enables instruction-following behavior.

Q63. What are GANs and their training challenges?

Generator and Discriminator trained adversarially. Challenges: 1) Mode collapse (generator produces limited variety). 2) Training instability (oscillations, divergence). 3) Vanishing gradients when discriminator is too good. Solutions: Wasserstein GAN loss, spectral normalization, progressive growing, balanced training.

Q64. What is the Transformer architecture?

Consists of encoder and/or decoder blocks. Each block: multi-head self-attention + position-wise feed-forward network + residual connections + layer normalization. Input: token embeddings + positional encodings. Attention enables O(n^2) but fully parallel token interactions. Foundation of BERT, GPT, T5, ViT.

Q65. What is positional encoding in transformers?

Since attention is permutation-invariant, we add positional encodings to token embeddings to inject sequence order. Original paper used sinusoidal functions of different frequencies. Modern models use learned positional embeddings or relative position encodings (RoPE, ALiBi) which generalize better to longer sequences.

Q66. Explain contrastive learning.

Self-supervised learning: train model so that similar pairs of inputs have close embeddings and dissimilar pairs have distant embeddings. No labels needed. SimCLR: augment same image twice (positive pair), all other images = negative pairs. CLIP: image-text pairs from internet. Produces powerful representations.

Q67. What is knowledge distillation?

Train a small student model to mimic a large teacher model. Student learns from soft probabilities (logits) of teacher, not just hard labels. Soft labels contain richer information about class relationships. Results in compact model with performance close to teacher. Used in DistilBERT (6x smaller, 97% of BERT performance).

Q68. What are graph neural networks (GNNs)?

Neural networks that operate on graph-structured data. Each node aggregates features from its neighbors (message passing). Applications: social networks, molecular property prediction, fraud detection, knowledge graphs. Key models: GCN, GAT (with attention), GraphSAGE (inductive).

Q69. What is quantization in deep learning?

Reducing model weight precision from 32-bit float to 8-bit integer (INT8) or even 4-bit. Drastically reduces model size and inference latency with minimal accuracy loss. Post-training quantization (PTQ): no retraining needed. Quantization-aware training (QAT): fine-tune with simulated quantization for better accuracy.

Q70. Explain multi-task learning.

Training one model on multiple related tasks simultaneously with shared parameters. Benefits: implicit regularization (tasks act as constraints), better generalization, fewer total parameters than separate models. Works best when tasks share useful representations. Example: BERT trained on MLM + NSP simultaneously.

Q71. What is Amazon's approach to ML (CRISP-DM)?

Amazon's ML process follows: Business Understanding -> Data Understanding -> Data Preparation -> Modeling -> Evaluation -> Deployment. Key aspect: start with the customer need and work backward (Working Backwards). Define clear success metrics tied to business outcomes before building.

Q72. Explain how AWS SageMaker works.

Fully managed ML platform. Key services: SageMaker Studio (IDE), Training Jobs (managed training on EC2), Automatic Model Tuning (hyperparameter optimization), Model Registry, Endpoints (real-time/batch inference), Feature Store, Pipelines (MLOps), Clarify (bias/explainability). Integrates with S3 for data storage.

Q73. What are the STAR method and behavioral questions for Amazon?

Situation-Task-Action-Result framework. Amazon's Leadership Principles are tested in behavioral interviews. Key ones for AS: Are Right A Lot, Invent and Simplify, Deep Dive, Bias for Action, Customer Obsession. Prepare 2-3 detailed stories per principle.

Q74. How does Microsoft Azure ML work?

Azure Machine Learning: managed platform with Designer (GUI), Automated ML (AutoML), Notebooks, Training Compute clusters, Model Registry, Endpoints. Integrates with Azure DevOps for MLOps. Uses Azure Blob Storage. Key products: Azure OpenAI Service, Cognitive Services.

Q75. What is MLOps?

DevOps practices applied to ML. Covers: data versioning, model versioning (MLflow), experiment tracking, CI/CD for models, automated testing, monitoring in production, automatic retraining triggers. Goal: reliable, reproducible, scalable ML in production.

Q76. How do you debug a model that is not converging?

1) Check learning rate (too high = divergence, too low = very slow). 2) Verify gradients are not NaN/exploding. 3) Check loss function matches task. 4) Verify data is normalized. 5) Start with tiny dataset (overfit first). 6) Use gradient clipping. 7) Try different optimizer or weight initialization.

Q77. What is early stopping?

Monitor validation loss during training. Stop training when validation loss stops improving (patience parameter). Prevents overfitting. Save best model checkpoint. Most frameworks have callbacks for this (Keras: EarlyStopping, PyTorch Lightning: ModelCheckpoint).

Q78. What is the difference between online learning and offline learning?

Offline (batch) learning: train once on static dataset, then deploy. Model does not update with new data. Simple but does not adapt to distribution shifts. Online learning: model updates incrementally with each new data point. Adapts to changes but requires careful monitoring to prevent degradation.

Q79. How do you handle temporal data / time series?

Key rule: never use future data in features (look-ahead bias). Time-based train/val/test split (not random). Features: lag features, rolling statistics, trend, seasonality. Models: ARIMA, Prophet for classical; LSTM, Temporal Fusion Transformer for deep learning. Evaluate with expanding window cross-validation.

Q80. What is confounding in causal inference?

A confounder is a variable that affects both the treatment (cause) and outcome, creating spurious correlation. Example: ice cream sales and drowning (both caused by hot weather). Causal inference methods: randomized experiments (gold standard), instrumental variables, difference-in-differences, propensity score matching.

Q81. What is the difference between precision and recall, and when to optimize for each?

Precision = TP/(TP+FP). Of predicted positives, how many are correct? Optimize when false positives are costly (spam filter, fraud alert). Recall = TP/(TP+FN). Of actual positives, how many did we catch? Optimize when false negatives are costly (cancer detection, security threat). F1 balances both.

Q82. What are embeddings and why are they important?

Dense vector representations that map discrete objects (words, items, users) to a continuous space where semantic similarity corresponds to vector proximity. Key: learned end-to-end. Enable neural networks to work with categorical data. Foundation of NLP (Word2Vec, BERT) and recommendations (collaborative filtering).

Q83. Explain SHAP values.

SHapley Additive exPlanations: assign each feature an importance value for a specific prediction based on game theory (Shapley values). Measures marginal contribution of each feature across all possible feature subsets. Model-agnostic, locally accurate, consistent. Better than permutation importance for correlated features.

Q84. What is the difference between hard and soft voting in ensembles?

Hard voting: each classifier votes for a class, majority wins. Soft voting: average the predicted probabilities across classifiers, pick class with highest average probability. Soft voting is generally better as it uses confidence information.

Q85. How does Adam optimizer work?

Adaptive Moment Estimation. Maintains: m = exponential moving average of gradients (1st moment = direction), v = exponential moving average of squared gradients (2nd moment = magnitude). Update: theta -= lr * m_hat / (sqrt(v_hat) + eps). Adapts learning rate per parameter. Typically lr=1e-3, beta1=0.9, beta2=0.999.

Q86. What is the difference between a validation set and a test set?

Validation set: used during development to tune hyperparameters and compare models. Can be used many times. Test set: final evaluation only. Must never be used during development or hyperparameter tuning. Touching the test set biases your estimate of generalization performance.

Q87. Explain the Bellman equation.

In RL, the Bellman equation expresses the value of a state as the immediate reward plus discounted future value: V(s) = max_a [R(s,a) + gamma * V(s')]. For Q-learning: Q(s,a) = R + gamma * max_a' Q(s', a'). Foundation of dynamic programming and Q-learning.

Q88. What is few-shot learning?

Training a model to generalize from very few labeled examples (1-shot: 1 per class, few-shot: 5-20). Approaches: 1) Meta-learning (MAML: learn good initializations). 2) Prototypical networks: classify by

proximity to class prototypes. 3) Prompt engineering with LLMs (in-context learning with examples in the prompt).

Q89. What is model calibration?

A model is well-calibrated if predicted probabilities match actual frequencies. E.g., among instances where model predicts 70% probability, ~70% should actually be positive. Measure with reliability diagrams. Fix with Platt scaling (logistic regression on outputs) or isotonic regression. Critical for risk-sensitive applications.

Q90. How would you approach a new ML problem from scratch?

1) Understand business goal + success metrics. 2) Explore data (EDA, distributions, correlations). 3) Establish baseline (simple heuristic or model). 4) Iterate: feature engineering, model selection, tuning. 5) Error analysis: understand failure modes. 6) Evaluate on test set. 7) Deployment plan + monitoring. Document everything.

Q91. What is the difference between semantic and syntactic similarity?

Syntactic: surface-level text similarity (word overlap, edit distance). 'The cat sat' vs 'A cat sat' = high syntactic similarity. Semantic: meaning-level similarity. 'The car stopped' vs 'The vehicle halted' = high semantic similarity. Modern NLP (BERT embeddings, sentence transformers) captures semantic similarity.

Q92. What is gradient clipping?

Caps the gradient norm during backpropagation to prevent exploding gradients, common in RNNs and deep networks. Two types: clip by value (clip each gradient component) or clip by norm (scale gradient vector if its norm exceeds threshold). Used in PyTorch: torch.nn.utils.clip_grad_norm_(params, max_norm=1.0).

Q93. What is the difference between instance segmentation and semantic segmentation?

Semantic segmentation: classify each pixel (all cats = one label). Instance segmentation: detect and segment each individual object separately (cat1, cat2 are distinct). Instance segmentation is harder and more useful. Semantic: FCN, DeepLab. Instance: Mask R-CNN (adds a segmentation head to Faster R-CNN).

Q94. How does beam search work in NLP generation?

Instead of greedily picking the most probable next token at each step, beam search maintains the top-B (beam width) most probable sequences at each step. Explores more of the search space than greedy but less than exhaustive search. Larger beam = better quality but slower. Common B=4 or 5 for translation.

Q95. What is label smoothing?

Instead of training with hard labels (0 or 1), use soft labels (e.g., 0.1 and 0.9). Prevents model from being overconfident, improves calibration and generalization. Equivalent to adding a small uniform

distribution to the target. Popular in image classification and NLP. Improves top-5 accuracy on ImageNet.

Q96. What is curriculum learning?

Inspired by human learning: present training examples in order from easy to hard. The model first learns simple patterns before tackling difficult ones. Can improve convergence speed and final performance. Used in NLP (short sentences first), RL (simple environments first), and self-supervised learning.

Q97. What is the difference between a generative model and a discriminative model?

Discriminative: models P(y|x) directly. Just learns the decision boundary. Examples: logistic regression, SVM, BERT for classification. Generative: models the joint distribution P(x, y) or P(x). Can generate new samples, handle missing data. Examples: Naive Bayes, VAE, GAN, GPT. Discriminative usually better for classification.

Q98. How would you improve a model that is already at good performance?

1) Better data: more labeled data or better labeling quality. 2) Ensemble: combine multiple models. 3) Larger model (if not resource-constrained). 4) Better features (domain expertise). 5) Error analysis: find patterns in mistakes. 6) Data augmentation. 7) Semi-supervised learning on unlabeled data. 8) Task-specific pre-training.

Q99. What is the PageRank algorithm?

Assigns importance scores to web pages based on the number and quality of links pointing to them. Modeled as a random walk on the web graph. Score of page = sum of (score of linking pages / number of their outgoing links). Iterative computation. Damping factor (0.85) models probability of following a link vs. jumping randomly.

Q100. Explain how FAISS works for approximate nearest neighbor search.

FAISS (Facebook AI Similarity Search) enables fast retrieval of nearest neighbors in high-dimensional spaces. Key methods: IVF (Inverted File Index -- cluster vectors, search only nearest clusters), PQ (Product Quantization -- compress vectors for memory efficiency), HNSW (hierarchical graph for fast search). Used in production recommendation systems.

Quick Reference: Key Formulas

```
# Bias-Variance:        Total Error = Bias^2 + Variance + Noise
# Precision:            TP / (TP + FP)
# Recall:               TP / (TP + FN)
# F1:                   2 * P * R / (P + R)
# Cross-Entropy:        -sum(y * log(p) + (1-y) * log(1-p))
# Softmax:              e^zi / sum(e^zj)
# Attention:            softmax(QK^T / sqrt(d_k)) * V
# Bayes:                P(A|B) = P(B|A) * P(A) / P(B)
# Gradient Descent:     theta = theta - lr * gradient(Loss)
# L2 Loss:              Loss = MSE + lambda * sum(w^2)
# L1 Loss:              Loss = MSE + lambda * sum(|w|)
```

Study Priority Checklist

- Master: Bias-Variance, Regularization, Loss Functions, Backprop, Gradient Descent
- Know well: Random Forest, XGBoost, SVM, Neural Network architectures (CNN, RNN, Transformer)
- Understand: PCA, K-Means, Evaluation Metrics, Cross-Validation
- Be able to code: Logistic Regression, K-Means, training loop in PyTorch
- System Design: Recommendation, Fraud Detection, Search Ranking frameworks
- Statistics: Hypothesis testing, A/B testing, Bayes' theorem, CLT